



Wrocław
University
of Science
and Technology

Software Engineering

Software architecture

Marcin Jodłowiec

November 5th, 2025

Agenda

- What is software architecture?
- Modularity, coupling and separation of concerns
- Logical vs. physical architecture
- Architectural styles
 - Classical architectures
 - Layered monolith vs modular monolith
 - Monolithic vs distributed
 - Distributed architecture
 - Service-oriented architecture
 - Event-driven architecture
 - Microservices architecture
 - Modeling software architecture

What is software architecture?

What is software architecture?

Architectural design

Architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system.

Software architecture

- ▶ It defines the **main structural components** of the system and the **relationships** between them.
- ▶ Acts as a **bridge between requirements and detailed design**.
- ▶ The result is an **architectural model**: a blueprint describing the system as a set of communicating components.

What is software architecture?

Laws of Software Architecture

1. Everything in software architecture is a trade-off.
2. *Why* is more important than *how*.
3. Most architecture decisions aren't binary but rather exist on a spectrum between extremes.

Mark Richards:

– *Architecture is the stuff you can't Google or ask an LLM about.*

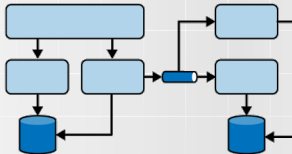
Example: Should we use queue-based communication or publish-subscribe model?

Modularity, coupling and separation of concerns

Modularity, Coupling, and Separation of Concerns I

Modularity

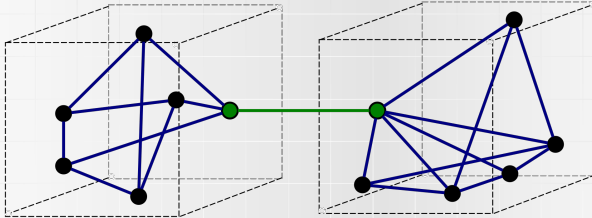
- ▶ The process of **decomposing a system** into smaller, manageable, and independent components.
- ▶ Each module encapsulates a coherent subset of system functionality.
- ▶ Enables **parallel development**, **comprehensibility**, and **reusability**.



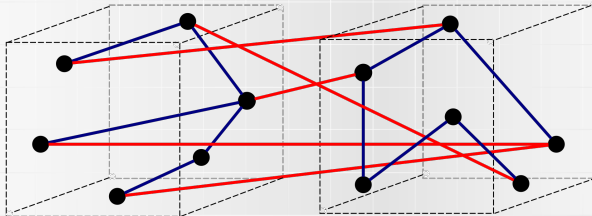
Coupling and Cohesion

- ▶ **Cohesion** – how strongly the elements within a module belong together. *High cohesion is desirable.*
- ▶ **Coupling** – the degree of interdependence between modules.
 - ▶ **Tight coupling:** modules are highly dependent on each other.
 - ▶ **Loose coupling:** modules have minimal dependencies.

Modularity, Coupling, and Separation of Concerns III



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

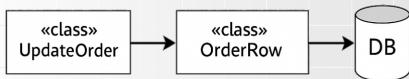
Source: Wikipedia

Modularity, Coupling, and Separation of Concerns IV

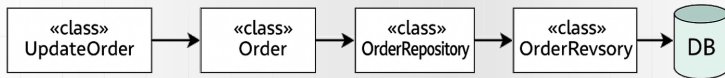
Separation of Concerns (SoC)

- ▶ Each module should address a **distinct concern**.
- ▶ Reduces complexity and facilitates independent evolution of system parts.
- ▶ Examples: UI vs. business logic, domain logic vs. persistence.

Without SoC:



With SoC:

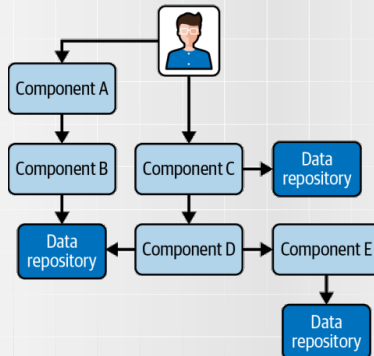


Logical vs. physical architecture

Logical vs. physical architecture I

Logical Architecture

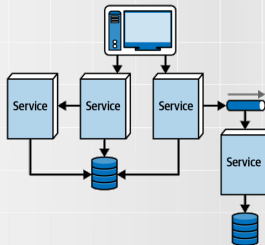
- ▶ Describes the system's logical components and how they interact.
- ▶ Focuses on functionality and responsibilities, not on deployment.
- ▶ Shows how components collaborate and exchange data.
- ▶ Independent of technologies and infrastructure.



Logical vs. physical architecture II

Physical Architecture

- ▶ Describes physical artifacts such as services, databases, APIs, and user interfaces.
- ▶ Focuses on deployment and runtime structure.
- ▶ Represents one of the architectural styles: layered, microservices, event-driven, etc.
- ▶ Maps logical components to deployable units (containers, services, executables).



Creating logical architecture

- ▶ Identifying Core Components
 - ▶ Workflow analysis – understand how users interact with the system.
 - ▶ Domain modeling – identify key entities and their relationships. Warning: entity trap.
 - ▶ The Actor/Action approach – identify main actors and their actions.
- ▶ Assigning User Stories to Components
- ▶ Analyzing roles and responsibilities
- ▶ Analyze architecture characteristics -scalability, reliability, availability, etc.
- ▶ Refactor, refine, add components as needed

Architectural styles

Architectural styles

Architectural style

- ▶ A **reusable solution** to a commonly occurring problem in software architecture within a given context.
- ▶ Provides a **template** for organizing system components and their interactions.
- ▶ Defines a set of **principles and patterns** that guide the design of the system.

Trade-offs: performance vs. scalability, complexity vs. maintainability, etc.

Big Ball of Mud

Definition

A **Big Ball of Mud** is a system with little or no recognizable architecture or structure. It typically evolves organically without clear modular boundaries or design principles.

Characteristics

- ▶ Event handlers and business logic directly coupled to database calls.
- ▶ No clear separation of concerns or layering.
- ▶ Hard to change, test, or scale.
- ▶ Architecture emerges accidentally rather than intentionally.

Consequences

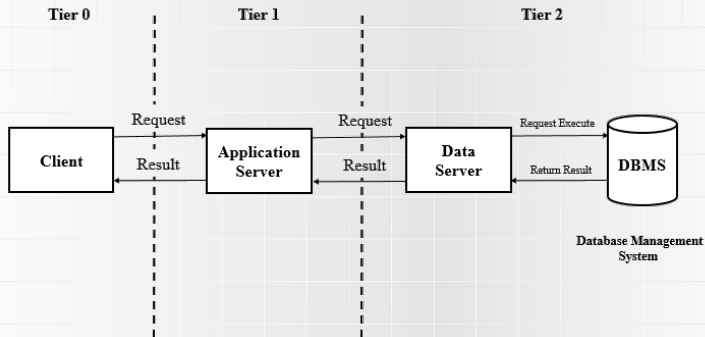
- ▶ High maintenance cost and fragile code base.
- ▶ Difficult deployment and poor performance.

Architectural styles

Foundational architectural concepts

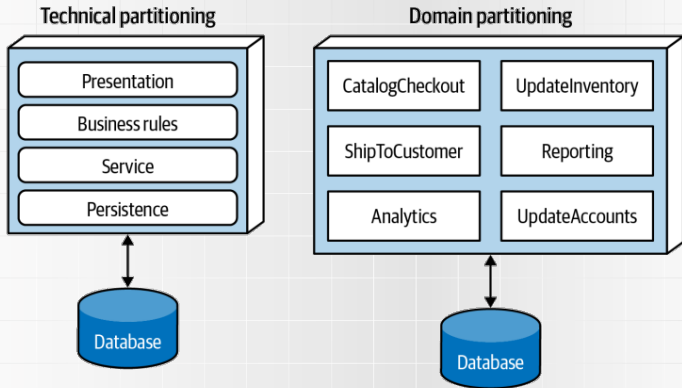
- ▶ Unitary architecture – all components run within a single process on single machine.
- ▶ Client-server architecture – distinct client and server components communicating over a network – separation of presentation logic and business logic.
 - ▶ Desktop and database servers.
 - ▶ Browser and web servers.
 - ▶ Single-page applications (SPAs).
 - ▶ Three-tier architecture (presentation, application, data).

Three-tier architecture



- ▶ Divides the system into three layers: **Presentation** (UI), **Application** (business logic), and **Data** (storage).
- ▶ Improves separation of concerns and maintainability; middle and presentation tiers can be scaled independently.

Layered monolith vs modular monolith



- ▶ **Layered monolith:** (*technical partitioning*) organizes code into layers (presentation, business logic, data access) but all layers are tightly coupled within a single deployable unit.
- ▶ **Modular monolith:** (*domain partitioning*) organizes code into independent modules with well-defined interfaces, allowing for better separation of concerns and easier maintenance while still being deployed as a single unit.

Monolithic vs distributed

Monolithic Architecture

- ▶ Single deployable unit containing all functionality
- ▶ Components share memory and resources
- ▶ Inter-component communication via direct method calls
- ▶ Simplified deployment and testing
- ▶ Easier to debug and trace

Distributed Architecture

- ▶ Multiple deployable units (services)
- ▶ Components run on different nodes
- ▶ Inter-component communication via network protocols
- ▶ Independent deployment and scaling
- ▶ Technology heterogeneity

Key Trade-offs

- ▶ **Monolithic:** Simpler development, faster local communication, but limited scalability and technology lock-in
- ▶ **Distributed:** Better scalability and fault isolation, but increased operational complexity and network latency

Distributed architecture

Fallacies of distributed computing

Reasons why distribution should be carefully taken (another trade-off to consider)

- ▶ The network is reliable.
- ▶ Latency is zero.
- ▶ Bandwidth is infinite.
- ▶ The network is secure.
- ▶ Topology doesn't change.
- ▶ There is one administrator.
- ▶ Transport cost is zero.
- ▶ The network is homogeneous.
- ▶ Versioning is easy.
- ▶ Observability is optional.

Service-oriented architecture I

Service-Oriented Architecture (SOA)

- ▶ A distributed architecture style emphasizing **reusable business services**.
- ▶ Services communicate via standardized protocols (SOAP, REST, messaging).
- ▶ Typically includes an **Enterprise Service Bus (ESB)** for integration and orchestration.
- ▶ Focus on **interoperability** and **enterprise-wide reuse**.

Service Taxonomy

- ▶ **Business services:** Coarse-grained services representing business functionality
- ▶ **Enterprise services:** Shared services across the organization
- ▶ **Application services:** Fine-grained services for specific applications
- ▶ **Infrastructure services:** Logging, monitoring, authentication, etc.

Service-oriented architecture II

Key Components

- ▶ **Service Registry:** Discovery mechanism for available services
- ▶ **Enterprise Service Bus (ESB):** Central hub for message routing, transformation, and orchestration
- ▶ **Orchestration Engine:** Coordinates multiple services (orchestration vs. choreography)
- ▶ **Service Contracts:** WSDL, API specifications defining service interfaces

Architecture Characteristics

- ▶ **Strengths:** High reusability, technology heterogeneity, organizational scalability
- ▶ **Weaknesses:** Complexity, performance overhead, ESB as single point of failure, slow change cycles, reuse implies coupling
- ▶ **Trade-off:** Reusability vs. simplicity and performance

Orchestration vs. Choreography

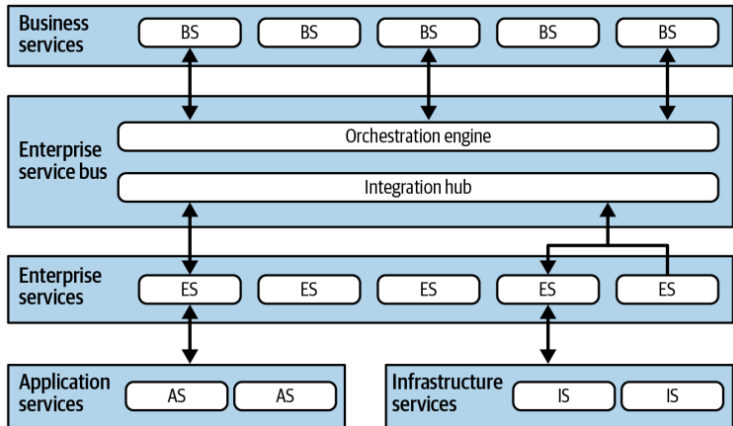
Orchestration

- ▶ A **central controller** (orchestrator) coordinates all service interactions.
- ▶ Each service performs tasks when explicitly instructed.
- ▶ The flow of control is **centrally managed**.
- ▶ Example: a workflow engine invokes services in a defined sequence.

Choreography

- ▶ There is **no central controller**.
- ▶ Each service reacts to events and triggers others by publishing new events.
- ▶ The flow of control is **distributed and event-driven**.
- ▶ Example: in an e-commerce system, the payment service emits an event that triggers shipping.

Service-oriented architecture



Event-driven architecture

Request-Based Model

- ▶ Synchronous communication
- ▶ Direct request-response
- ▶ Tight coupling between components
- ▶ Immediate response expected
- ▶ Blocking operations

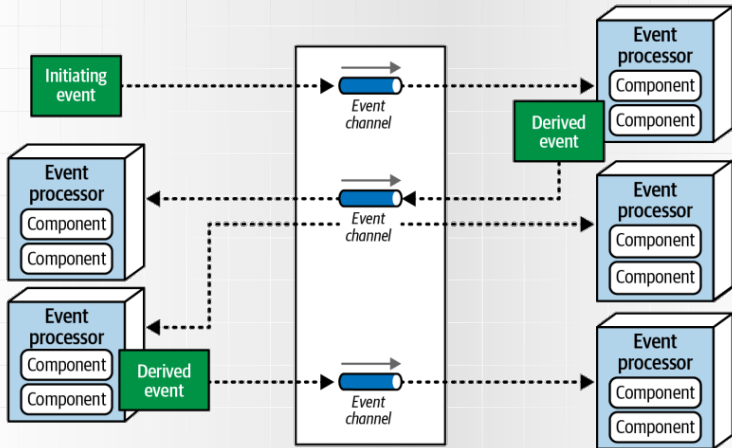
Event-Driven Architecture

- ▶ Asynchronous communication
- ▶ Publish-subscribe pattern
- ▶ Loose coupling between components
- ▶ No immediate response required
- ▶ Non-blocking operations

Key Characteristics of EDA

- ▶ **Events** are the primary communication mechanism
- ▶ Components publish events when significant things happen
- ▶ Other components subscribe and react to events
- ▶ Enables reactive, real-time systems
- ▶ Supports scalability and fault tolerance

EDA Topology



Architecture Trade-offs

- ▶ **Advantages:** Loose coupling, scalability, real-time responsiveness
- ▶ **Disadvantages:** Eventual consistency, debugging complexity, event ordering challenges
- ▶ **Best for:** Systems requiring high responsiveness and decoupling

Microservices architecture

Microservices Architecture

- ▶ An architectural style that structures an application as a collection of **loosely coupled services**.
- ▶ Each service is **independently deployable** and **scalable**.
- ▶ Services use **lightweight protocols** (HTTP/REST, messaging).
- ▶ Each service owns its own **data and domain logic**.
- ▶ Enables **technology diversity** and **independent evolution**.

Service Characteristics

- ▶ **Small and focused:** Each service addresses a specific business capability
- ▶ **Independent:** Services can be deployed without affecting others
- ▶ **Technology agnostic:** Different services can use different technologies
- ▶ **Resilient:** Failure in one service doesn't bring down the entire system
- ▶ **Scalable:** Individual services can be scaled independently

Key Components

- ▶ **API Gateway:** Single entry point for client requests
- ▶ **Service Registry/Discovery:** Dynamic service location and registration
- ▶ **Configuration Management:** Centralized configuration for all services

Microservices: Communication & Trade-offs

Communication

- ▶ **Synchronous:** Direct request-response (REST, gRPC)
- ▶ **Asynchronous:** Message queues, event streaming for decoupling
- ▶ Services can use different protocols based on needs

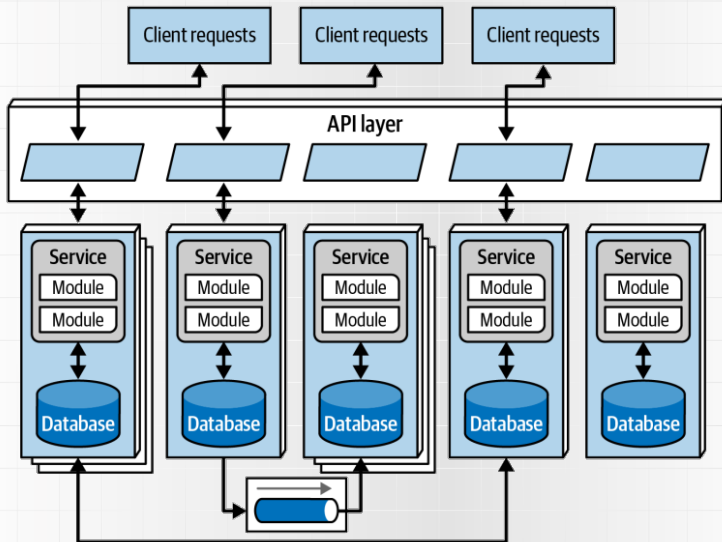
Data Management

- ▶ **Database per service:** Each service owns its data store
- ▶ **Eventual consistency:** Data consistency across services handled via events
- ▶ Distributed data requires careful design and coordination

Architecture Characteristics

- ▶ **Strengths:** Scalability, technology flexibility, independent deployment, fault isolation
- ▶ **Weaknesses:** Operational complexity, distributed system challenges, testing difficulty, data consistency
- ▶ **Trade-off:** Agility and scalability vs. complexity and operational overhead

Microservices architecture



Modeling software architecture

UML (Unified Modeling Language)

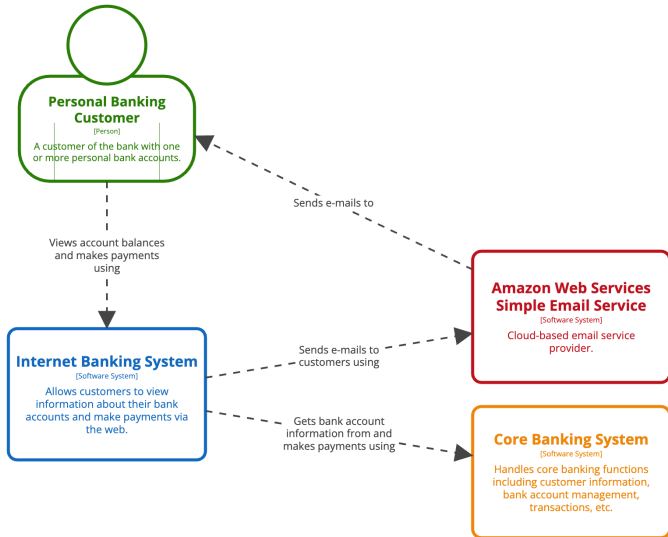
- ▶ Unified competing modeling notations into a single standard.
- ▶ Widely used for **class** and **sequence diagrams**, but most other types are rarely used today.
- ▶ Often considered heavyweight and overly formal for agile environments.
- ▶ For architecture modeling, **Component Diagrams** (logical) and **Deployment Diagrams** (physical) are most relevant.
 - ▶ Detailed info during structural modeling lecture :-)

C4 Model

- ▶ Introduced by Simon Brown (2006–2011) as a modern alternative to UML.
- ▶ Useful for distributed architectures.
- ▶ Defines four hierarchical levels of abstraction:
 - ▶ **Context** – system and its external actors.
 - ▶ **Container** – applications, databases, and services within the system.
 - ▶ **Component** – internal building blocks of each container.
 - ▶ **Class (Code)** – detailed code-level structures (optional).
- ▶ Lightweight, tool-friendly, and widely adopted in modern software architecture.

C4 Model

Context

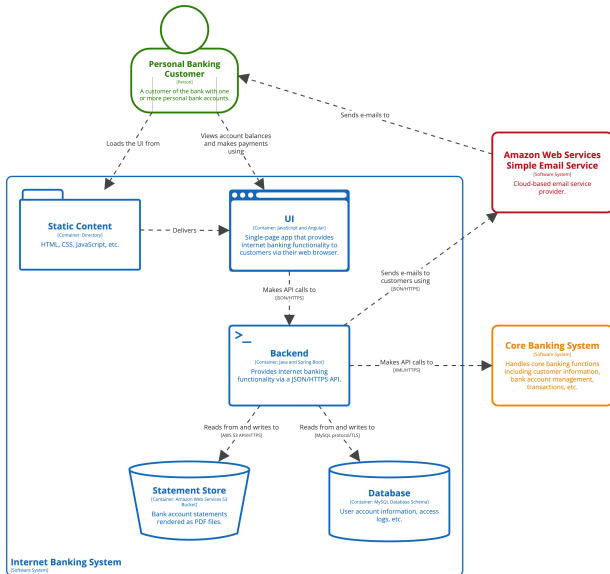


System Context View: Internet Banking System

The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

C4 Model

Containers

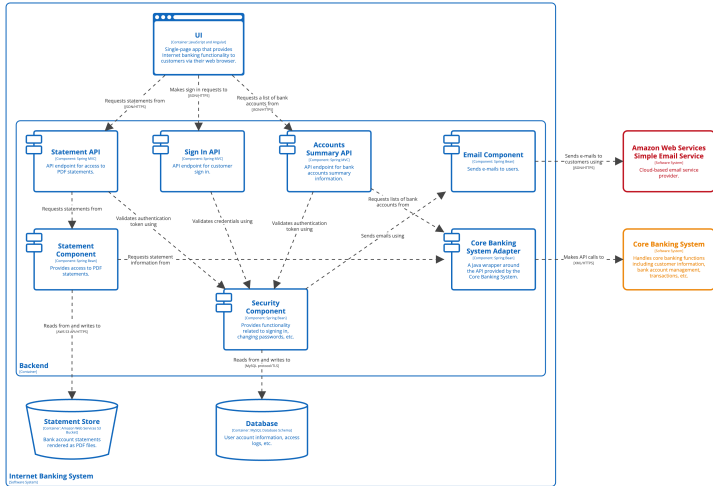


Container View: Internet Banking System

The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

C4 Model

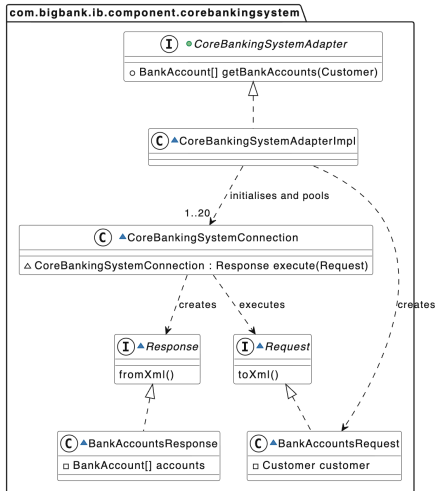
Components



Component View: Internet Banking System - Backend

The component diagram for the Internet Banking System Backend | Simon Brown | olnesd.com | License: CC BY 4.0

C4 Model Code



Code View: Internet Banking System - Backend - Core Banking System Adapter

A summary of the implementation details for the Core Banking System Adapter component | Simon Brown | c4model.com | License: CC BY 4.0

It's all for today!

